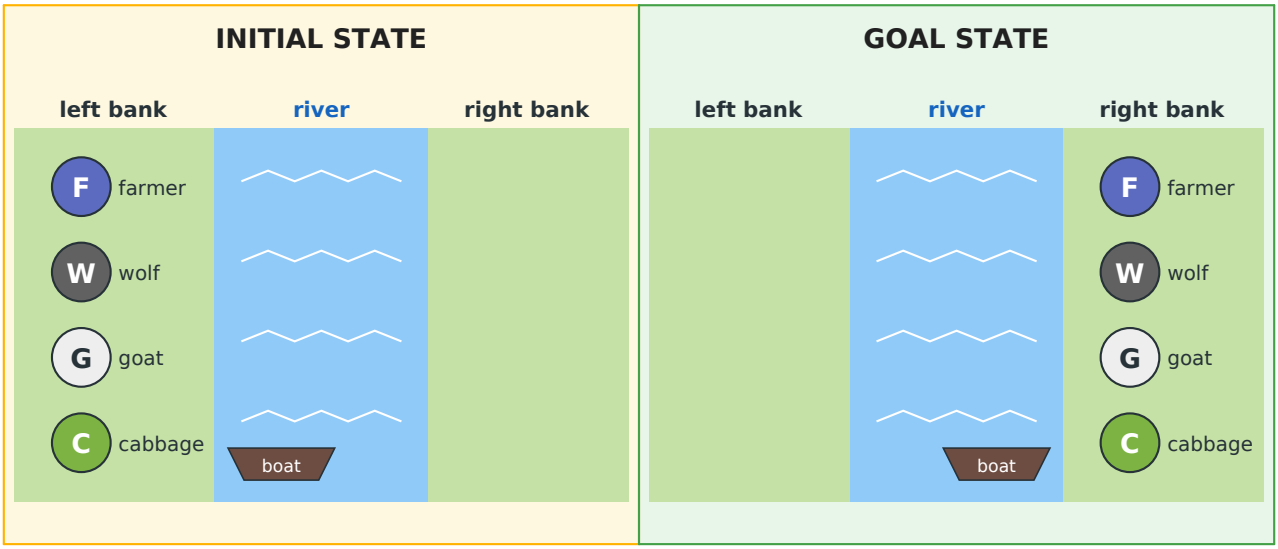
Problem 7: Farmer, Wolf, Goat and Cabbage

Level: **Intermediate** · Folder: **07_Farmer_Wolf_Goat_Cabbage/** · Files: domain.pddl, problem.pddl, solution_plan.txt

1. The problem

A farmer is on the left bank of a river with a **wolf**, a **goat** and a **cabbage**. The boat holds the farmer and at most **one** passenger. If left alone, the wolf eats the goat and the goat eats the cabbage. Get everyone safely to the right bank.



The goal picture shows one state that satisfies the goal (the final state of the plan). Only the facts in the :goal section are required; anything else may be different.

What you will learn

- Writing **safety constraints** as preconditions.
- Working without negative preconditions by splitting an action into **safe variants**.
- Using a static relation (*opposite*) so that one action works in both directions.
- Plans that need 'backward' steps (the goat is carried back).

2. Formulating the problem in PDDL

Step 1: Types and objects. One type bank; objects left, right. Each character has its own predicate.

Step 2: Predicates describe what can be true or false in a state:

Predicate	Meaning
`(farmer-at ?b)`	The farmer (and the boat) is on bank ?b.
`(wolf-at ?b)`	The wolf is on bank ?b.
`(goat-at ?b)`	The goat is on bank ?b.
`(cabbage-at ?b)`	The cabbage is on bank ?b.
`(opposite ?b1 ?b2)`	?b2 is across the river from ?b1 (static).

Step 3: Actions. Each action has *parameters*, a *precondition* (what must be true to apply it) and an *effect* (facts that become **true (+)** or **false (−)**):

Action	Precondition	Effect

cross-alone-goat-away
`?from - bank, ?to - bank`
Farmer crosses alone. Safe because the goat is already on the other side.
	`(opposite ?from ?to)`
	`(farmer-at ?from)`
	`(goat-at ?to)`
		`+ (farmer-at ?to)`
		`− (farmer-at ?from)`

cross-alone-goat-alone
`?from - bank, ?to - bank`
Farmer crosses alone. The goat stays, but wolf and cabbage are both on the other side.
	`(opposite ?from ?to)`
	`(farmer-at ?from)`
	`(goat-at ?from)`
	`(wolf-at ?to)`
	`(cabbage-at ?to)`
		`+ (farmer-at ?to)`
		`− (farmer-at ?from)`

cross-with-goat
`?from - bank, ?to - bank`
Farmer takes the goat. Always safe, because wolf and cabbage can be left together.
	`(opposite ?from ?to)`
	`(farmer-at ?from)`
	`(goat-at ?from)`
		`+ (farmer-at ?to)`
		`+ (goat-at ?to)`
		`− (farmer-at ?from)`
		`− (goat-at ?from)`

cross-with-wolf-goat-away
`?from - bank, ?to - bank`
Farmer takes the wolf. The goat is already across.
	`(opposite ?from ?to)`
	`(farmer-at ?from)`
	`(wolf-at ?from)`
	`(goat-at ?to)`
		`+ (farmer-at ?to)`
		`+ (wolf-at ?to)`
		`− (farmer-at ?from)`
		`− (wolf-at ?from)`

cross-with-wolf-cabbage-away
`?from - bank, ?to - bank`
Farmer takes the wolf. The cabbage is already across.
	`(opposite ?from ?to)`
	`(farmer-at ?from)`
	`(wolf-at ?from)`
	`(cabbage-at ?to)`
		`+ (farmer-at ?to)`
		`+ (wolf-at ?to)`
		`− (farmer-at ?from)`
		`− (wolf-at ?from)`

cross-with-cabbage-goat-away
`?from - bank, ?to - bank`
Farmer takes the cabbage. The goat is already across.
	`(opposite ?from ?to)`
	`(farmer-at ?from)`
	`(cabbage-at ?from)`
	`(goat-at ?to)`
		`+ (farmer-at ?to)`
		`+ (cabbage-at ?to)`
		`− (farmer-at ?from)`
		`− (cabbage-at ?from)`

cross-with-cabbage-wolf-away
`?from - bank, ?to - bank`
Farmer takes the cabbage. The wolf is already across.
	`(opposite ?from ?to)`
	`(farmer-at ?from)`
	`(cabbage-at ?from)`
	`(wolf-at ?to)`
		`+ (farmer-at ?to)`
		`+ (cabbage-at ?to)`
		`− (farmer-at ?from)`
		`− (cabbage-at ?from)`

Step 4: Initial state (`:init`) lists *every* fact that is true at the start. Everything else is false (closed-world assumption). **Step 5: Goal** (`:goal`) lists the facts that must be true at the end.

INITIAL STATE (6 facts)	GOAL

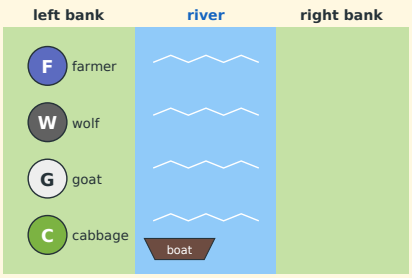
`(opposite left right)`
`(opposite right left)`
`(farmer-at left)`
`(wolf-at left)`
`(goat-at left)`
`(cabbage-at left)`
	`(farmer-at right)`
	`(wolf-at right)`
	`(goat-at right)`
	`(cabbage-at right)`

Modelling notes

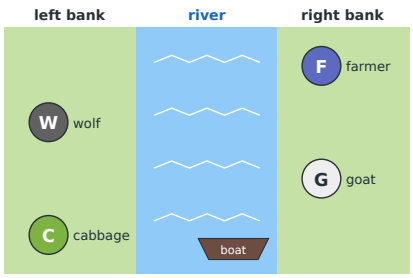
- **Key rule:** only the bank the farmer *leaves* can become dangerous. The bank the farmer arrives at is supervised. Each variant lists one situation in which the bank left behind is safe.
- With the ADL requirements `:negative-preconditions` and `:disjunctive-preconditions` you could write a single action, e.g. `(not (and (wolf-at ?from) (goat-at ?from)))`. Not every planner supports this, so the STRIPS version is more portable.
- **opposite** lets every action work in both directions (left to right and right to left).

3. Solution plan (7 actions, optimal)

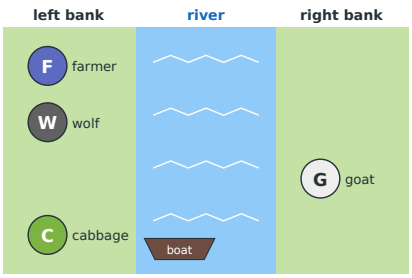
The goat is carried over first, carried back in step 4, and carried over again at the end (7 crossings). The pictures below show the state **after** each action. They were produced by executing the plan on the PDDL model.



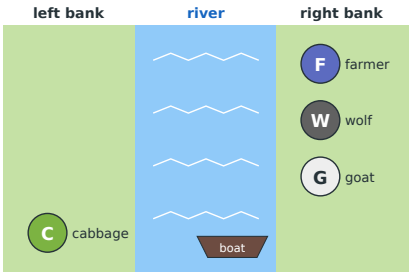
Initial state



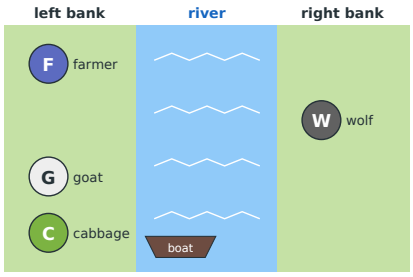
Step 1
`(cross-with-goat left right)`



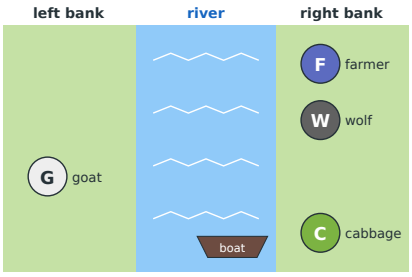
Step 2
`(cross-alone-goat-alone right left)`



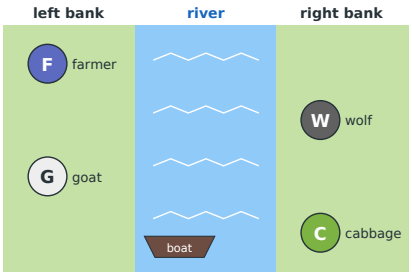
Step 3
`(cross-with-wolf-goat-away left right)`



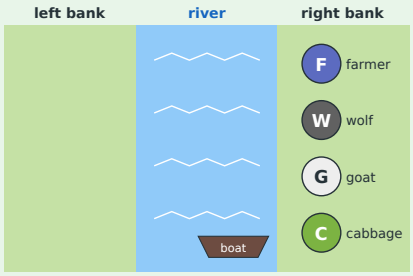
Step 4
`(cross-with-goat right left)`



Step 5
`(cross-with-cabbage-wolf-away left right)`



Step 6
`(cross-alone-goat-away right left)`



Step 7
`(cross-with-goat left right)`

4. How to run it

Online (no installation)	Open **editor.planning.domains**, paste *domain.pddl* and *problem.pddl* into two tabs and press **Solve**.
pyperplan (Python)	`pip install pyperplan` `pyperplan domain.pddl problem.pddl` (breadth-first search) `pyperplan -s astar -H lmcut domain.pddl problem.pddl` (optimal A*)
Fast Downward	`fast-downward.py domain.pddl problem.pddl --search "astar(lmcut())"`

5. Lab exercises

1. There is a second optimal solution (take the cabbage before the wolf). Find it by hand and check it with the planner by adding a goal or removing an action.
2. Rewrite the domain with :negative-preconditions and :disjunctive-preconditions using 4 actions, and test it on editor.planning.domains.
3. Model **Missionaries and Cannibals** (3 + 3, boat for 2) in the same style.

6. PDDL source code (appendix)

domain.pddl

```
;; ============================================================
;; Problem 7 : Farmer, Wolf, Goat and Cabbage (river crossing)
;; Level     : Intermediate
;; The boat carries the farmer plus at most one passenger.
;; Never leave the wolf alone with the goat, or the goat alone
;; with the cabbage.
;;
;; We use plain STRIPS (no negative preconditions), so each
;; "safe" situation is written as a separate action variant.
;; The rule: the bank the farmer LEAVES must stay safe.
;; ============================================================
(define (domain river-crossing)
  (:requirements :strips :typing)

  (:types bank)

  (:predicates
    (farmer-at ?b - bank)
    (wolf-at ?b - bank)
    (goat-at ?b - bank)
    (cabbage-at ?b - bank)
    (opposite ?b1 - bank ?b2 - bank)  ; ?b2 is across the river from ?b1
  )

  ;; ---- farmer crosses alone ---------------------------------
  ;; safe if the goat is already on the other bank
  (:action cross-alone-goat-away
    :parameters (?from - bank ?to - bank)
    :precondition (and (opposite ?from ?to) (farmer-at ?from)
                       (goat-at ?to))
    :effect (and (farmer-at ?to) (not (farmer-at ?from)))
  )
  ;; safe if the goat stays, but wolf and cabbage are both away
  (:action cross-alone-goat-alone
    :parameters (?from - bank ?to - bank)
    :precondition (and (opposite ?from ?to) (farmer-at ?from)
                       (goat-at ?from) (wolf-at ?to) (cabbage-at ?to))
    :effect (and (farmer-at ?to) (not (farmer-at ?from)))
  )

  ;; ---- farmer takes the goat (always safe: wolf+cabbage ok) ---
  (:action cross-with-goat
    :parameters (?from - bank ?to - bank)
    :precondition (and (opposite ?from ?to) (farmer-at ?from)
                       (goat-at ?from))
    :effect (and (farmer-at ?to) (goat-at ?to)
                 (not (farmer-at ?from)) (not (goat-at ?from)))
```

```
  )

  ;; ---- farmer takes the wolf (goat must not stay with cabbage)
  (:action cross-with-wolf-goat-away
    :parameters (?from - bank ?to - bank)
    :precondition (and (opposite ?from ?to) (farmer-at ?from)
                       (wolf-at ?from) (goat-at ?to))
    :effect (and (farmer-at ?to) (wolf-at ?to)
                 (not (farmer-at ?from)) (not (wolf-at ?from)))
  )
  (:action cross-with-wolf-cabbage-away
    :parameters (?from - bank ?to - bank)
    :precondition (and (opposite ?from ?to) (farmer-at ?from)
                       (wolf-at ?from) (cabbage-at ?to))
    :effect (and (farmer-at ?to) (wolf-at ?to)
                 (not (farmer-at ?from)) (not (wolf-at ?from)))
  )

  ;; ---- farmer takes the cabbage (wolf must not stay with goat)
  (:action cross-with-cabbage-goat-away
    :parameters (?from - bank ?to - bank)
    :precondition (and (opposite ?from ?to) (farmer-at ?from)
                       (cabbage-at ?from) (goat-at ?to))
    :effect (and (farmer-at ?to) (cabbage-at ?to)
                 (not (farmer-at ?from)) (not (cabbage-at ?from)))
  )
  (:action cross-with-cabbage-wolf-away
    :parameters (?from - bank ?to - bank)
    :precondition (and (opposite ?from ?to) (farmer-at ?from)
                       (cabbage-at ?from) (wolf-at ?to))
    :effect (and (farmer-at ?to) (cabbage-at ?to)
                 (not (farmer-at ?from)) (not (cabbage-at ?from)))
  )
)
```

problem.pddl

```
;; Everyone starts on the left bank; everyone must reach the right bank.
(define (problem river-crossing-1)
  (:domain river-crossing)

  (:objects left right - bank)

  (:init
    (opposite left right) (opposite right left)
    (farmer-at left)
    (wolf-at left)
    (goat-at left)
    (cabbage-at left)
  )

  (:goal (and (farmer-at right)
              (wolf-at right)
              (goat-at right)
              (cabbage-at right)))
)
```

solution_plan.txt

```
1. (cross-with-goat left right)
2. (cross-alone-goat-alone right left)
3. (cross-with-wolf-goat-away left right)
4. (cross-with-goat right left)
5. (cross-with-cabbage-wolf-away left right)
6. (cross-alone-goat-away right left)
7. (cross-with-goat left right)
```